Noah HONG MAYERS

A26-247GFBLG

Programmation Temps Réel

LABORATOIRE PID

Travail présenté à Francis **LAUZON DURANCEAU**

Collège Lionel-Groulx 10 Novembre 2026

cd ~/documents/LABS/247-GFB-LG-AUT2026-LAB-005/conduite-pid//open project

.\SimulateurVoiture.exe --qualite basse --fps 30 -screen-fullscreen 0 -screen-width 800 -screen-height 450//open low res simulator small window

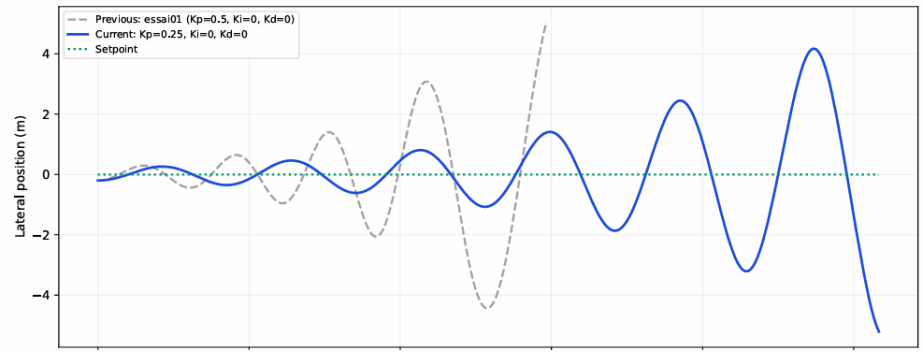
cmake --build build //build code

./build/conduite 172.28.176.1 5000 P I D essai0# //runs simulator link

Essai 1:

Nous sommes sur une piste droite avec un écart initial de 0.2m. (considérez que cela sera le cas jusqu'à tant que je mentionne explicitement un changement)

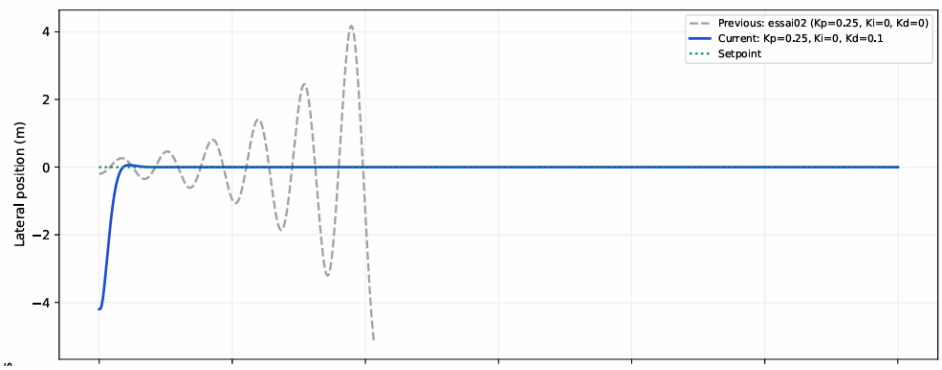
Avec un gain P de 0.25, la voiture oscille sans arrêt puisque son gain est proportionnel à l'erreur. Chaque dépassement entraine une plus grande correction dans le sens inverse. Sans autre facteur de correction, le degré correction augmente sans cesse. Ma prochaine initiative sera d'implémenter un gain dérivé pour que la voiture sache quand elle se rapproche du centre et corriger son gain en fonction.



Essai 2:

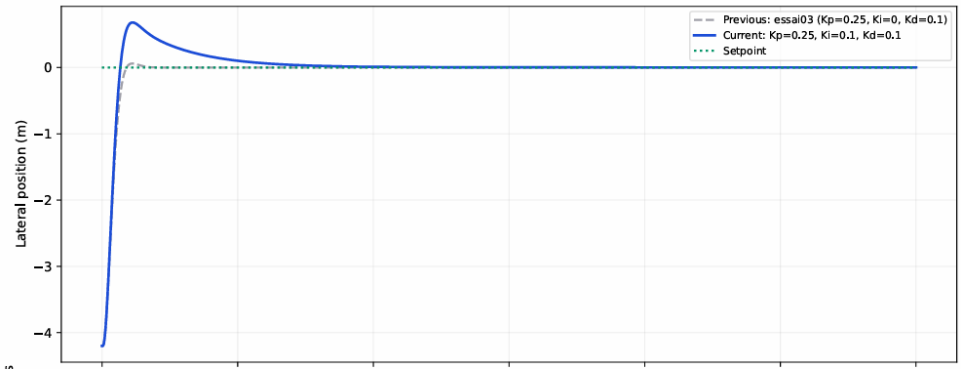
Nous sommes maintenant sur un écart initial de 4.2m.

En ajoutant un gain dérivé de 0.1 on remarque immédiatement que l'oscillation disparait. Le gain dérivé permet à la voiture de se situer en relation au centre et de diminuer son gain lorsqu'il s'en approche. Ma prochaine expérimentation sera d'ajouter le gain intégral.



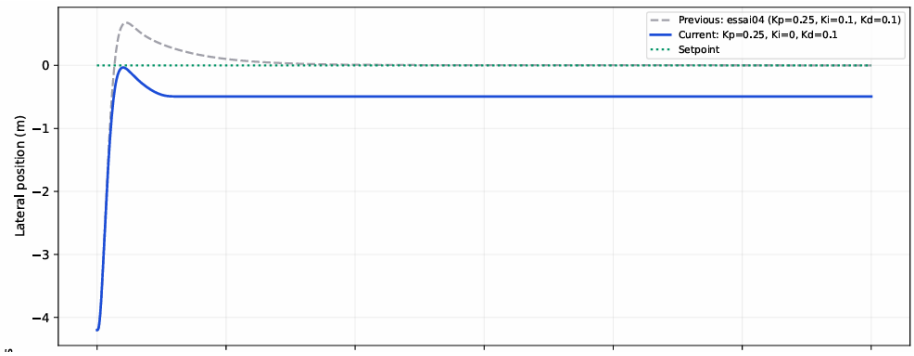
Essai 3:

En ajoutant le gain intégral nous remarquons qu'en fait notre temps de stabilisation à empiré. D'emblée, c'est étrange. Le gain intégral permet de corriger les erreurs qui s'accumules et augmenter le gain s'il y a une accumulation d'erreur que le P et le D ne peuvent pas corriger tout seul. Dans notre cas ici c'est exactement pour cette raison qu'il y a un plus gros dépassement au début. Le gain intégral a vu l'accumulation brève d'erreur au début et, en conséquence, a tenté de corriger plus agressivement. Le gain intégral est utile quand il y a une force ou un facteur externe constant qui pousse notre système à se stabiliser dans un état décalé. C'est l'erreur statique. Mon prochain test va démontrer ce principe.



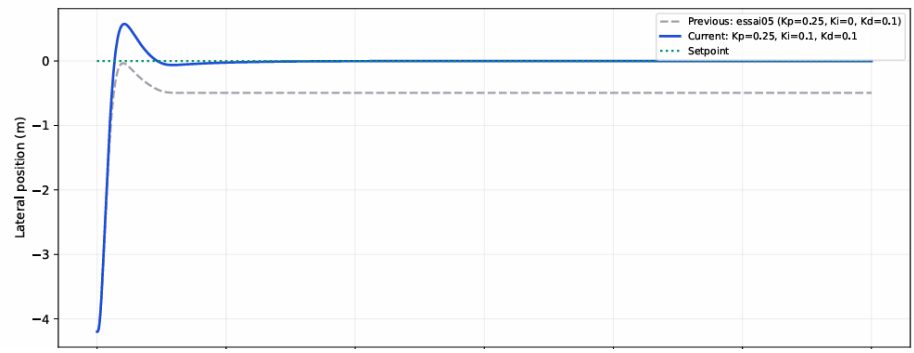
Essai 4:

Nous sommes maintenant sur une courbe constante pour simuler une perturbation extérieure qui empêche la voiture de se centrer. Nous avons enlevé le gain intégral pour démontrer l'effet. La voiture est constamment en train de tourner et donc ne se stabilise jamais réellement autour du centre. On le voit ici dans le décalage. Nous allons maintenant rajouter le I.



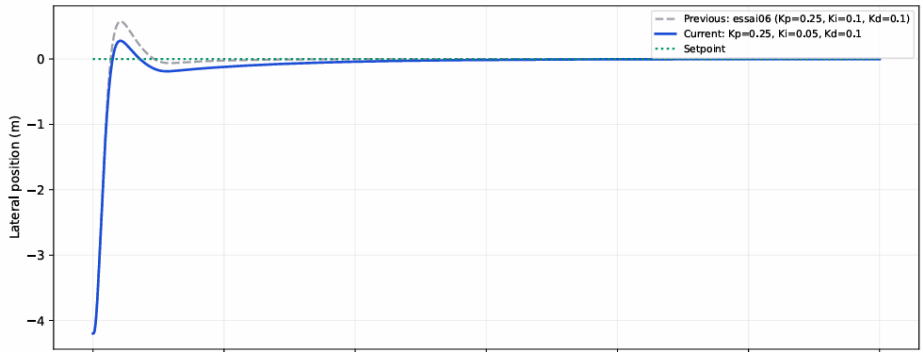
Essai 5:

Le gain intégral est maintenant de 0.1. Comme prévu, le gain intégral permets au système de voir l'accumulation d'erreur statique causée par la courbe et pousser une correction plus forte pour centrer la voiture. Nous sommes maintenant témoins d'un PID entièrement intégré. On remarque quand même un dépassement au début. Nous allons tenter de diminuer le gain intégral afin de voir si ce dépassement s'atténue.



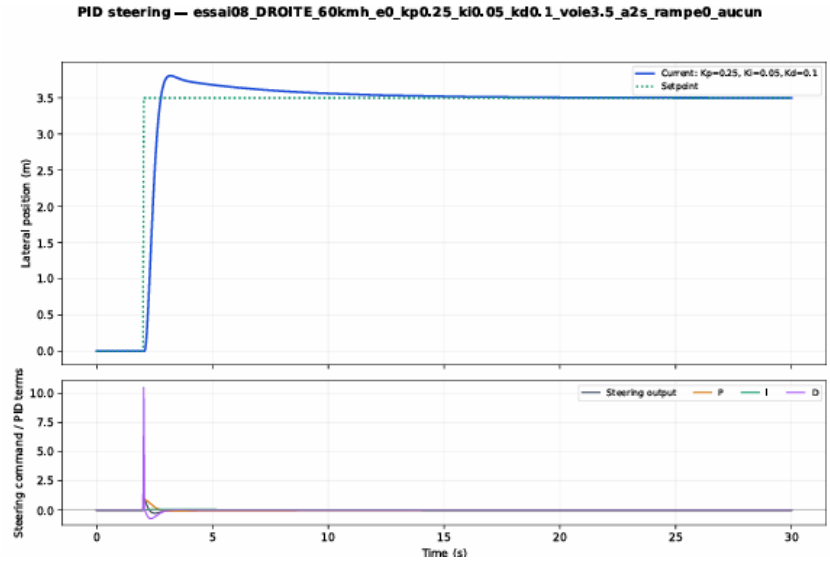
Essai 6:

Le gain intégral est maintenant de 0.05. En effet, le dépassement c'est atténué car la correction due a l'accumulation d'erreur initiale est moins forte, mais pour la même raison, le système prend plus longtemps à se corriger dans la courbe qui suit.

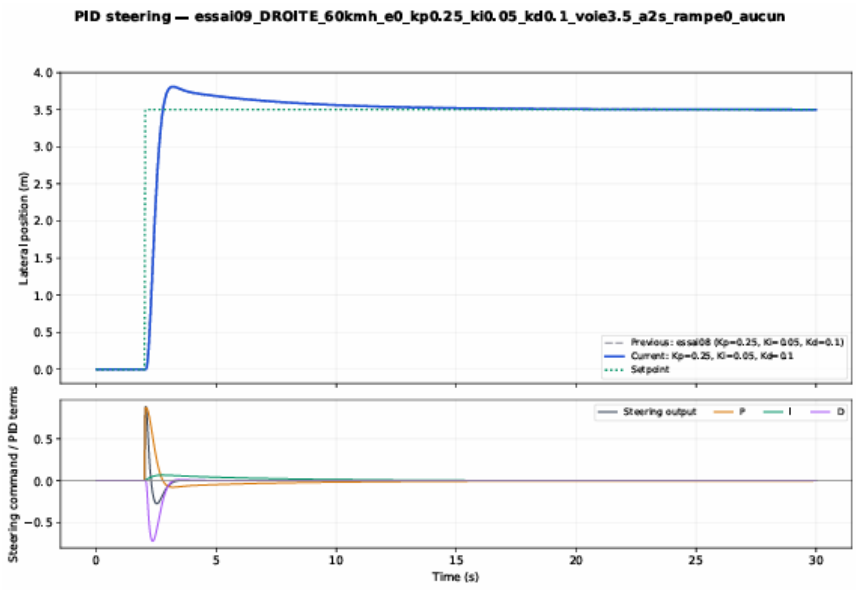


Étape 3: Experiences

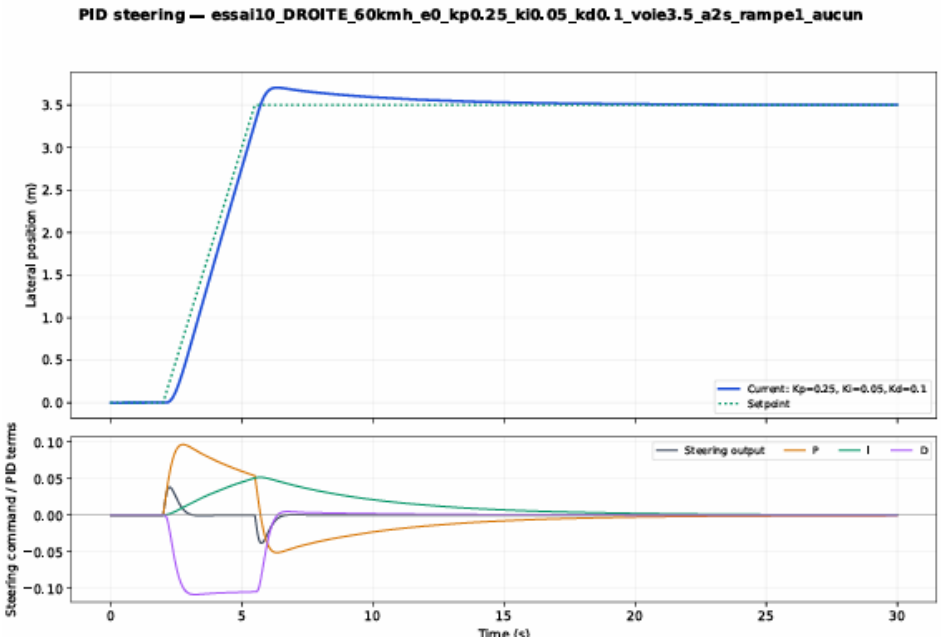
Gain dérivé calculé sur l'erreur. On remarque un grand pique dans D. Notre gain dérivé est proportionnel à la variation de l'erreur qui ici est très grande d'un coup. Ont défini le gain dérivé comme étant un gain résistif proportionnel à la vitesse de notre voiture. Calculer la vitesse avec l'erreur fonctionne désormais seulement si la consigne ne change pas puisqu'une erreur qui diminue donnera un gain négatif. Ici la consigne change brusquement ce qui incite D de croire que la voiture bouge très vite et en s'éloignant de la consigne ce qui cause un énorme gain positif. Nous avons limité le gain final à 1 et -1, donc cet énorme gain en D n'affecte pas notre courbe finale.



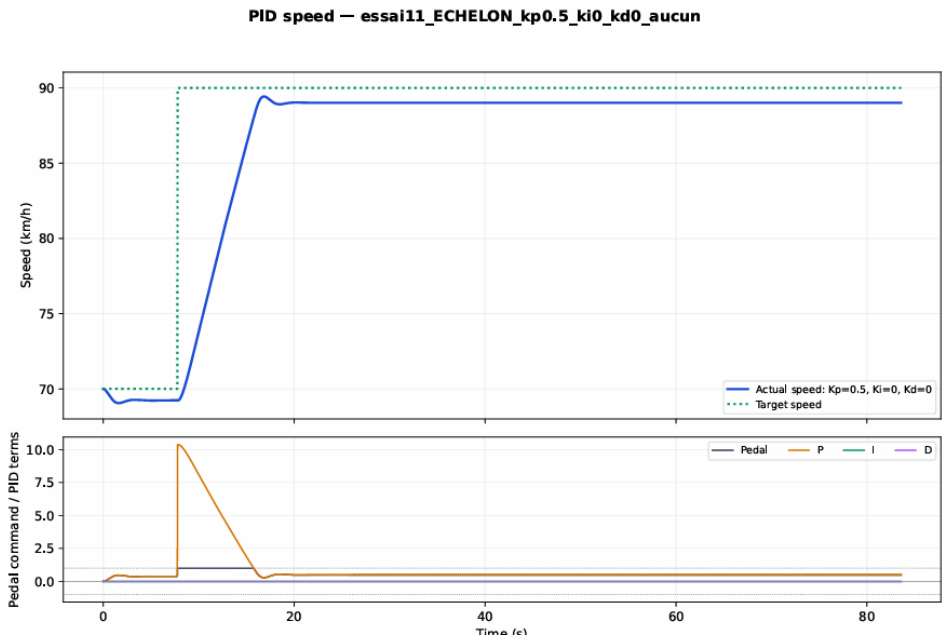
Gain dérivé calculé sur la mesure de position. Ici le gain est proportionnel à la vitesse de variation de la position de la voiture. Puisque le changement de la consigne n'affecte pas la position de la voiture, le gain dérivé n'est pas en enjeu jusqu'à ce que la voiture commence à bouger pour de vrai. Cela évite le pique dans D qu'on a vu tantôt.



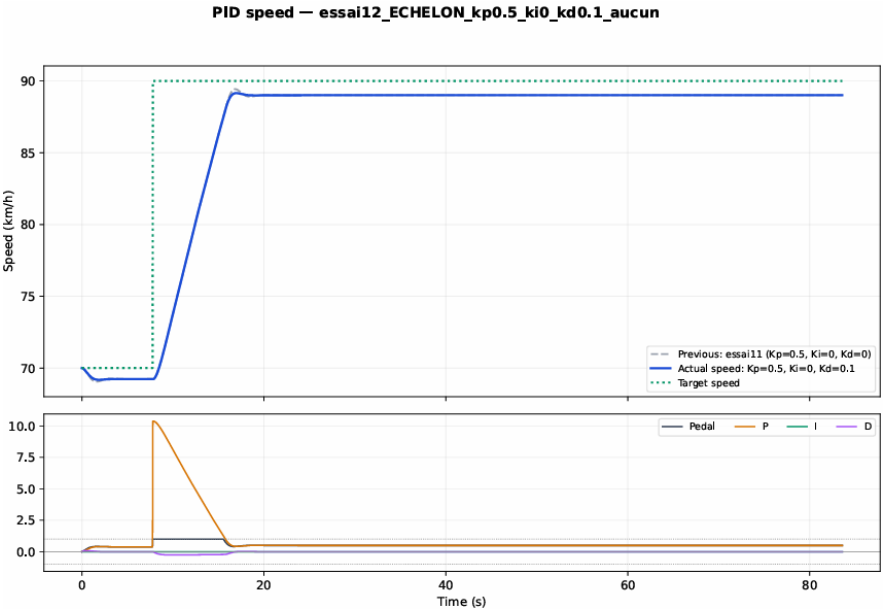
Changement de la consigne de façon graduelle. Ici la consigne change graduellement au lieu de la faire instantanément. On remarque alors que D affecte un gain résistif pendant l'entièreté du changement de la consigne puisque la voiture est tout le temps en mouvement latéral durant la pente. On remarquera aussi le I qui remarque l'accumulation de l'erreur et tente de la corriger.



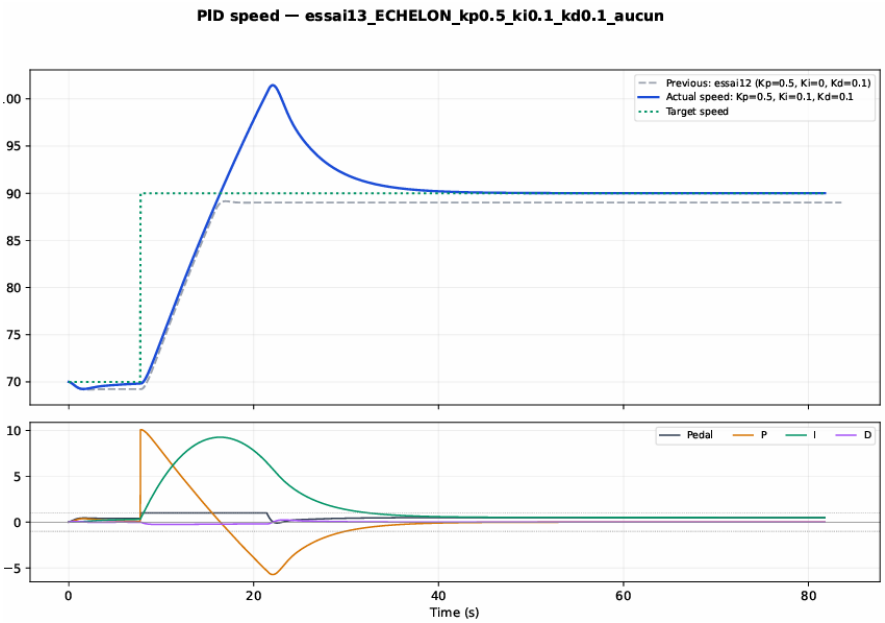
On essaiera juste un gain P à 0.1. Lorsque la vitesse augmente, le P ajoute un gain et le véhicule accélère. Il se stabilise juste en dessous de la consigne. J'estime que cette erreur statique est causée par la résistance naturelle qui oppose la voiture. On remarquera aussi un petit dépassement avant que la voiture se stabilise. On ajoutera maintenant un gain en D pour prévenir cela.



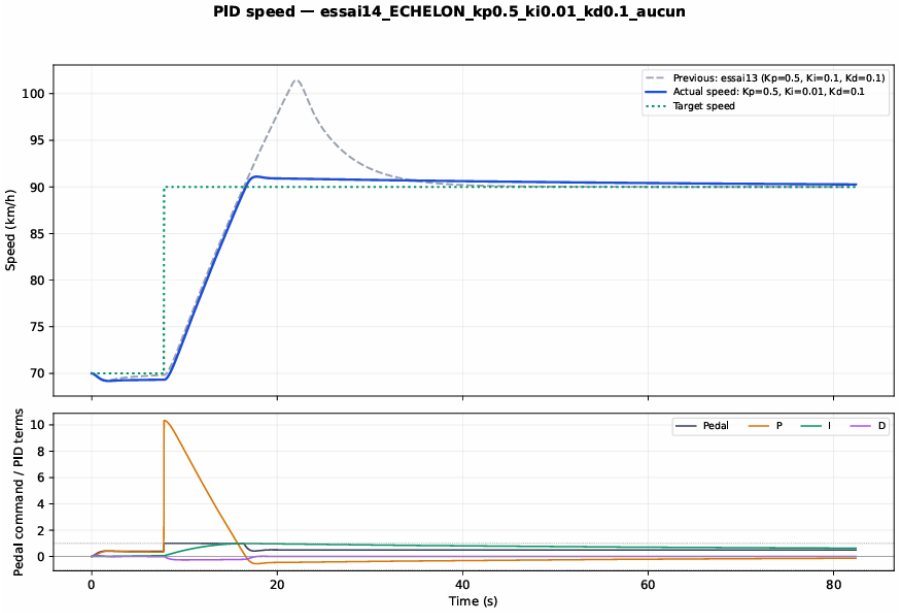
Avec l'ajout d'un gain en D de 0.1, la voiture ralentit lorsqu'elle se rapproche de la consigne et donc ne dépasse pas autant. Il y a quand même le problème de l'erreur statique.



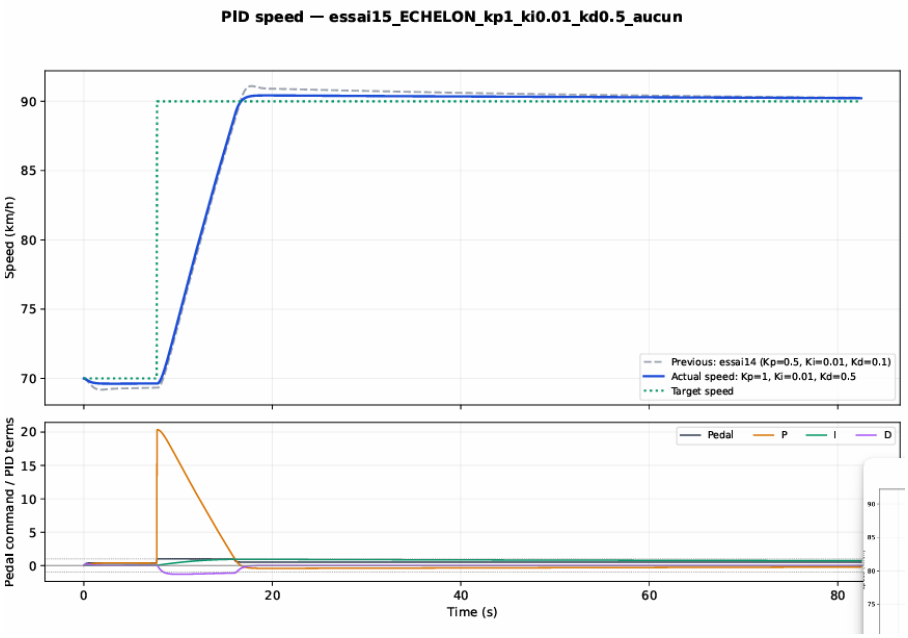
Nous avons ajouté une gain en I de 0.1. On remarque que l'erreur statique s'est corrigé, mais il y a un énorme dépassement due à l'accumulation d'erreur avant de se rendre à la consigne. On dimuera alors le I.



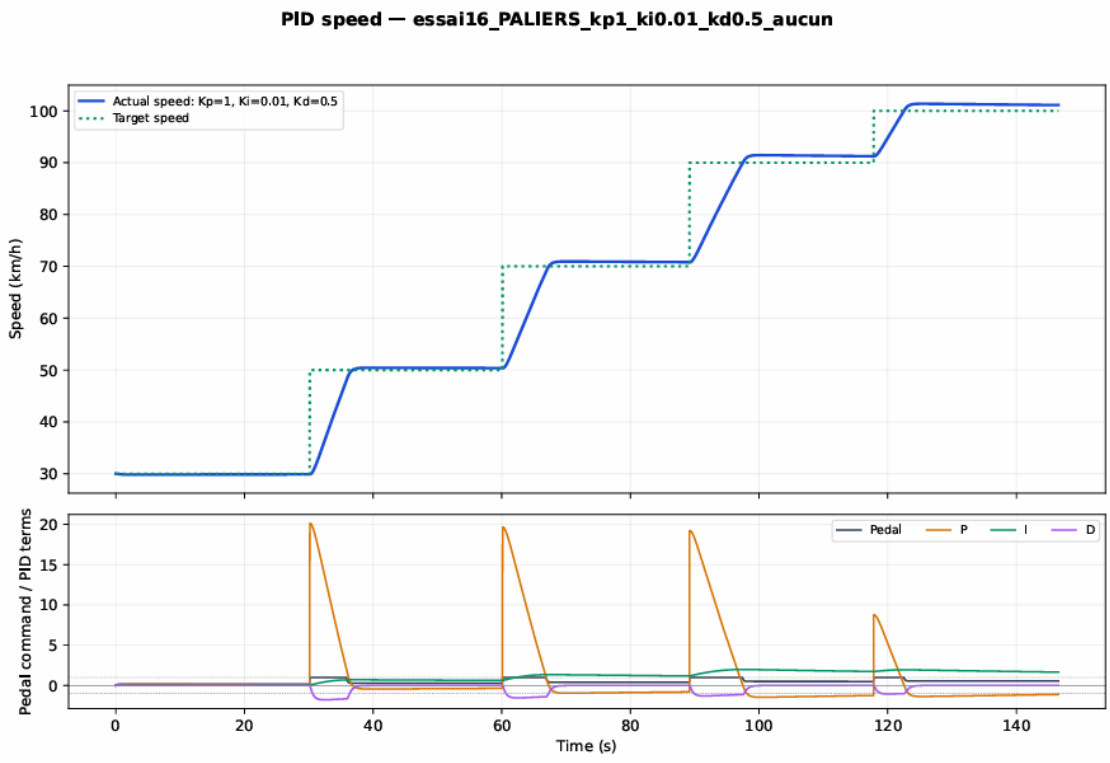
Nous avons réduit le I à 0.01. L'erreur statique se stabilise beaucoup plus lentement, mais nous avons éliminé le dépassement. On peut encore améliorer en augmentant le gain en D pour réduire le dépassement initial d'avantage.



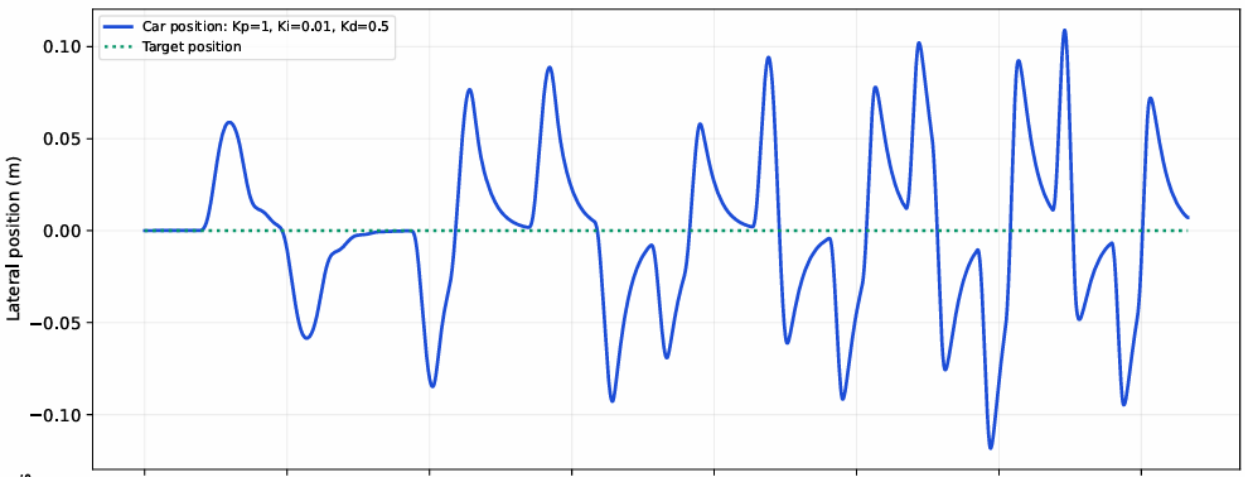
Le D est maintenant de 0.5. On voit alors que le dépassement s'est réglé et la courbe se stabilise tranquillement vers le consigne grâce au I.



En appliquant cette configuration aux paliers de vitesse. On retrouve étrangement une erreur de dépassement qui semble augmenter à chaque palier. Mon hypothèse est que les étapes de vitesses ne laissent pas le temps au I de corriger l'erreur accumulé. L'erreur accumulé lors de la monté de vitesse devient énorme et n'est pas entièrement corrigé avant que la prochaine étape de vitesse arrive ce qui fait en sorte que les erreurs accumulées non corrigés s'empilent.



Au niveau de la conduite de la voiture, un effet similaire existe. Au début quand les virages sont moins rapides et moins fréquents, la voiture arrive à se corriger. Lorsqu'on augmente la fréquence et la vitesse, la correction est plus violente et n'arrive pas complètement à se corriger avant la prochaine courbe.



1. Quelle tâche écrit sur le lien, et pourquoi une seule ControlTask écrit sur le lien.

```
// L'etat de la boucle, sur les lignes 5 et 6 de l'ecran. L'ecran a son propre
// mutex.
void ControlTask::show(int tic, double position, double consigne, double volant)
{
    if (ecran == nullptr) {
        return;
    }
    const char* etat = "      ";
    if (pid.requested() > kVolantMax || pid.requested() < kVolantMin) {
        etat = "SATURE";
    }
    char texte[100];
    std::snprintf(texte, sizeof(texte), "tic %6d   t %6.1f s   position %7.3f m   volant %6.3f   %s",
                  tic, tempsS, position, volant, etat);
    ecran->printAt(1, 5, texte);
    std::snprintf(texte, sizeof(texte), "P %7.3f   I %7.3f   D %7.3f   consigne %5.2f m   retards %d",
                  pid.termP(), pid.termI(), pid.termD(), consigne, lien.LateWarnings());
    ecran->printAt(1, 6, texte);
}
```

Lorsque ControlTask écrit, il appelle la méthode printAt() qui vérouille le mutex de l'objet 'ecran' de la classe Screen pour que personne ne puisse écrire sur l'écran en même temps et créer des erreurs d'affichages.

```
void Screen::printAt(int colonne, int ligne, const char* texte) {
    mutex.take();   // une seule tâche écrit à la fois
    printf("\033[%d;%dH%s", ligne, colonne, texte);
    fflush(stdout);
    mutex.release();
}
```

2. Comment une touche du clavier se rend jusqu'au Pid de la tâche de commande

Le code écoute pour une touche du clavier et l'envoie vers HandleKey

```
bool fini = false;
while (!fini) {
    if (clavier.kbhit(100)) {
        fini = HandleKey(clavier.getch(), reglages, ecran);
    }
}
```

HandleKey crée une copie des réglages dans 'r' et appel ApplyKey.

```
// Une touche : lire les reglages, les modifier, les ecrire d'un bloc.
// Retourne vrai pour quitter.
bool HandleKey(char touche, SharedSettings& reglages, Screen& ecran)
{
    if (touche == 'q') {
        return true;
    }
    Settings r = reglages.read();
    if (ApplyKey(touche, r)) {
        reglages.write(r);
        ShowSettings(ecran, r);
    }
    return false;
}
```

ApplyKey décide quoi faire selon la touche. Selon la touche, elle modifie la copie 'r'. La fonction retourne 1 s'il y a u un changement.

```
// Applique la touche a la copie r. Retourne vrai si un reglage a change.
// Un gain ou la protection qui change commence aussi un nouvel essai.
bool ApplyKey(char touche, Settings& r)
{
    bool change = true;
    switch (touche) {
        case 'p': r.kp = HalveOrZero(r.kp, 0.01); break;
        case 'P': r.kp = DoubleOrStart(r.kp, 0.1); break;
        case 'i': r.ki = HalveOrZero(r.ki, 0.01); break;
        case 'I': r.ki = DoubleOrStart(r.ki, 0.1); break;
        case 'd': r.kd = HalveOrZero(r.kd, 0.01); break;
        case 'D': r.kd = DoubleOrStart(r.kd, 0.05); break;
        case '1': r.voieM = -3.5; break;
        case '2': r.voieM = 0.0; break;
        case '3': r.voieM = 3.5; break;
        case 'a': r.antiWindup = NextAntiWindup(r.antiWindup); break;
        case 'n': r.numeroEssai = r.numeroEssai + 1; break;
        default: change = false; break;
    }
    if (ChangesGains(touche)) {
        r.numeroEssai = r.numeroEssai + 1;
    }
    return change;
}
```

Une fois la copie traité par ApplyKey, on écrit la copie des réglages aux vrais réglages dans l'objet SharedSettings.

```
bool HandleKey(char touche, SharedSettings& reglages, Screen& ecran)
{
    if (touche == 'q') {
        return true;
    }
    Settings r = reglages.read();
    if (ApplyKey(touche, r)) {
        reglages.write(r);
        ShowSettings(ecran, r);
    }
    return false;
}
```

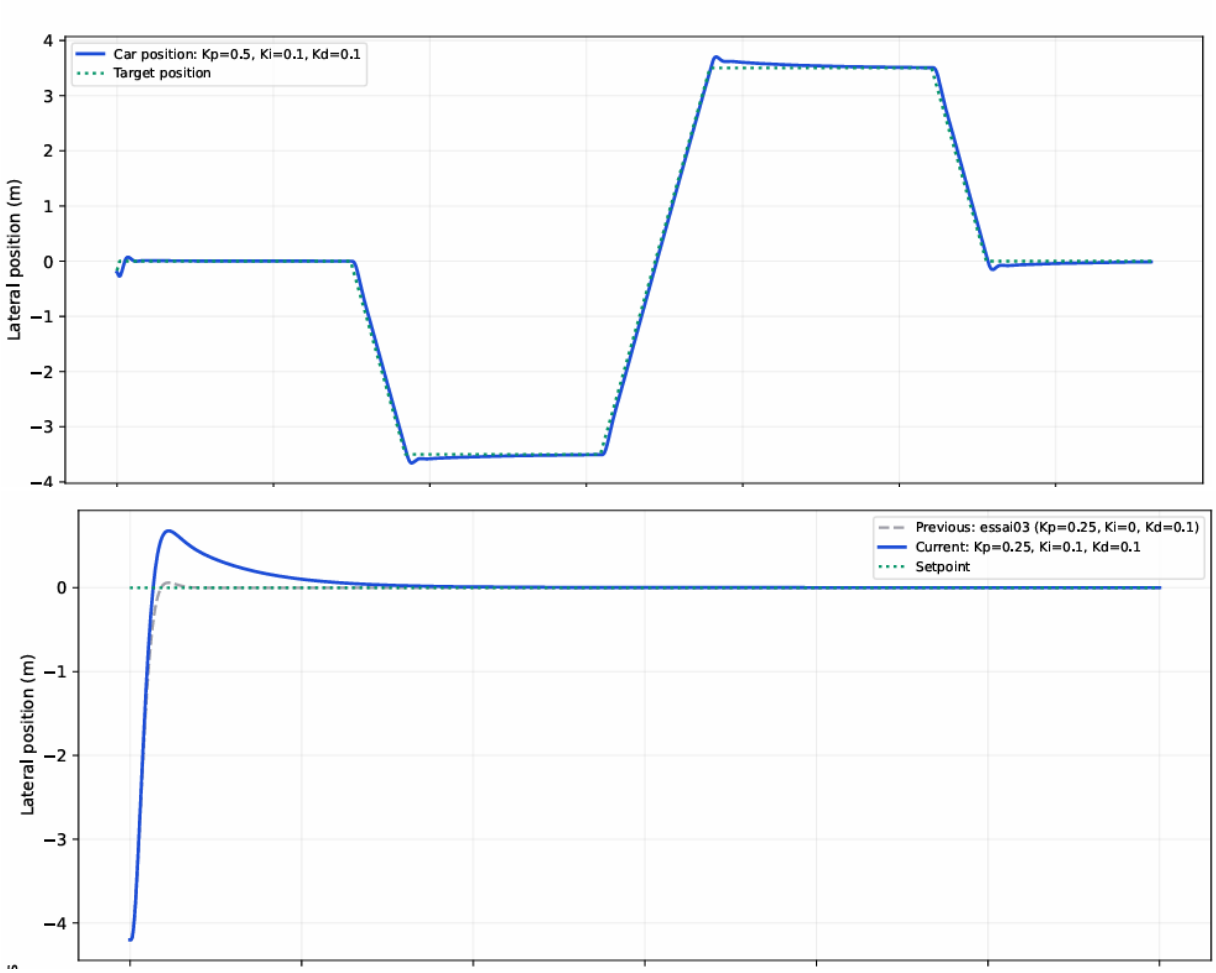
Sa méthode write verrouille son mutex pour éviter la concurrence d'écriture et les lectures durant l'écriture.

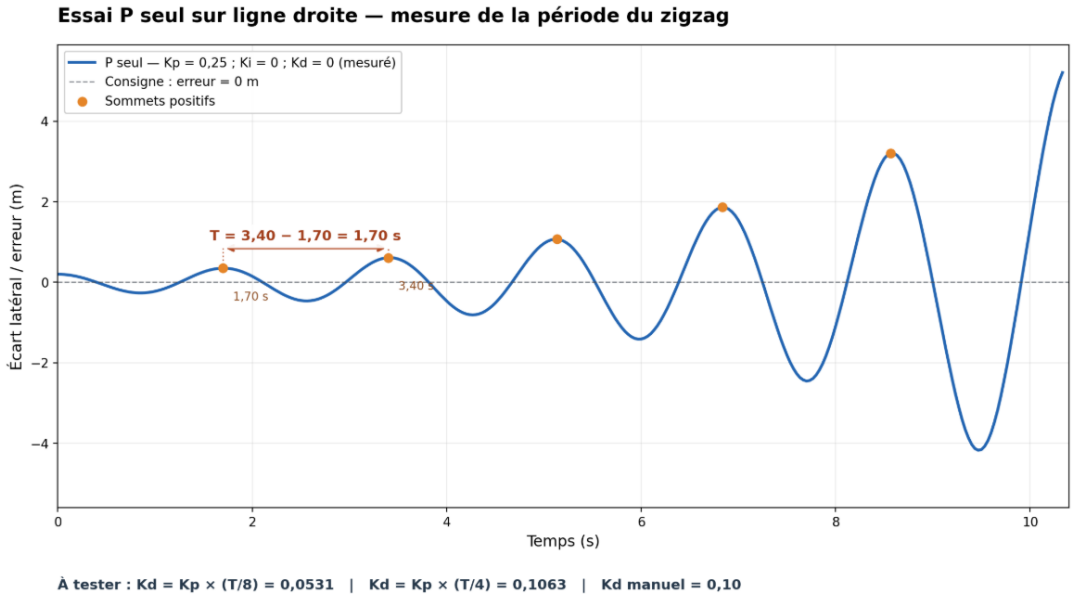
```
void SharedSettings::write(const Settings& nouveaux)
{
    mutex.take();
    reglages = nouveaux;
    mutex.release();
}
```

2. Pourquoi cmake -- tick() lit tous les réglages une seule fois, au début du tic, plutôt que chaque gain au moment de s'en servir.

   On lit tous les réglages en même temps de façon synchronisée afin d'éviter de les lire dans un **état instable**. Par exemple si on lit les réglages de façon indépendante, on pourrait demander de lire **pendant** qu'une modification s'effectue. On aurait donc un mélange des anciens gains et des nouveaux.

Le graphique d’en haut à été fait avec conduite_taches.
Le graphique d’en bas est celui de la partie 2.
On remarque que l’efficacité du PID demeure similaire. Il y a peut-être un plus grand dépassement dans le deuxième graphique puisque le changement de consigne est plus brusque et donc le I compense en trop.





$$T = 1.70s$$

$$Kd = Kp \,.\frac{T}{8}$$

$$Kd(doubled) = Kp \,.\frac{T}{4}$$

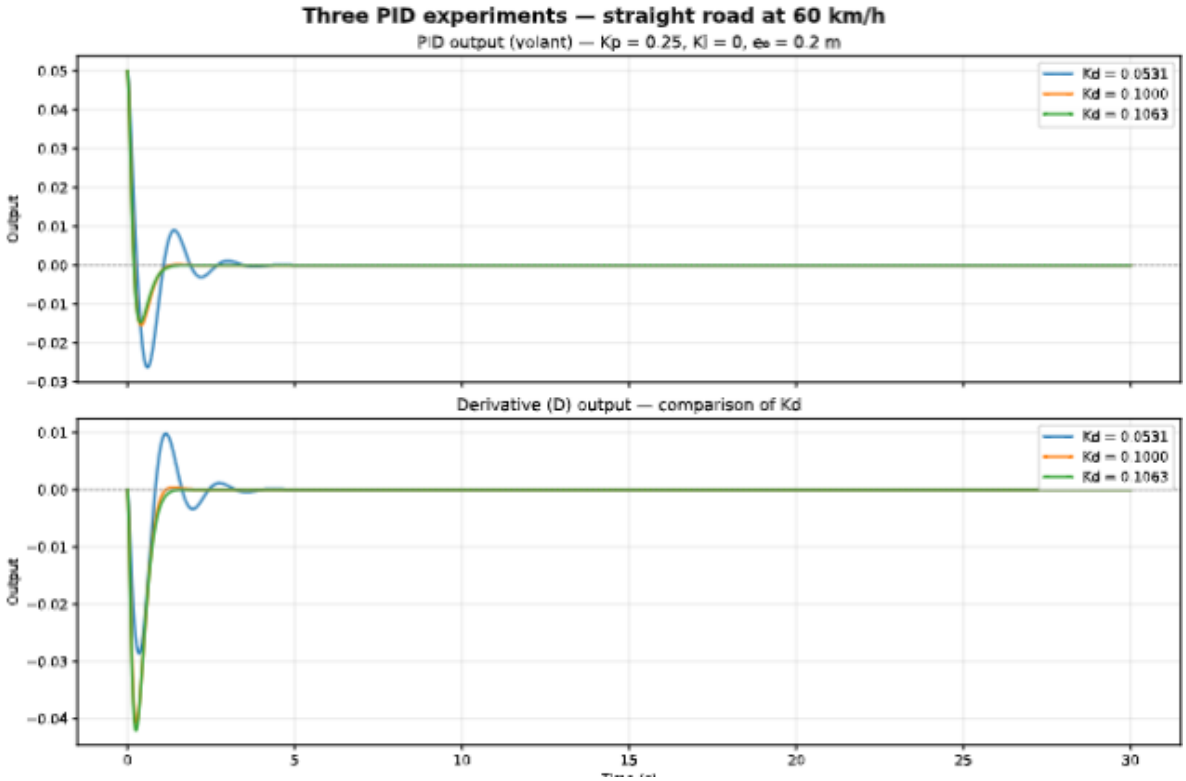
$$\boldsymbol{Kd = 0.0531}$$

$$\boldsymbol{Kd(doubled) = 0.1063}$$

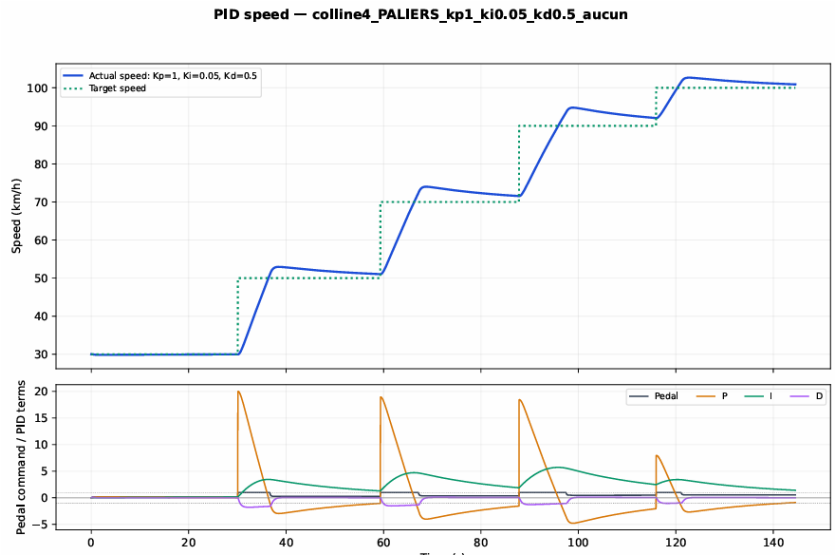
0.05131 nous donne de l’oscillation réduite, mais toujours apparente, car le gain dérivé n’est pas assez grand pour ralentir le mouvement lorsque la voiture s’approche de la consigne.

0.1063 retire entièrement l’oscillation et ralentit la voiture assez pour qu’elle ne dépasse pas la consigne.

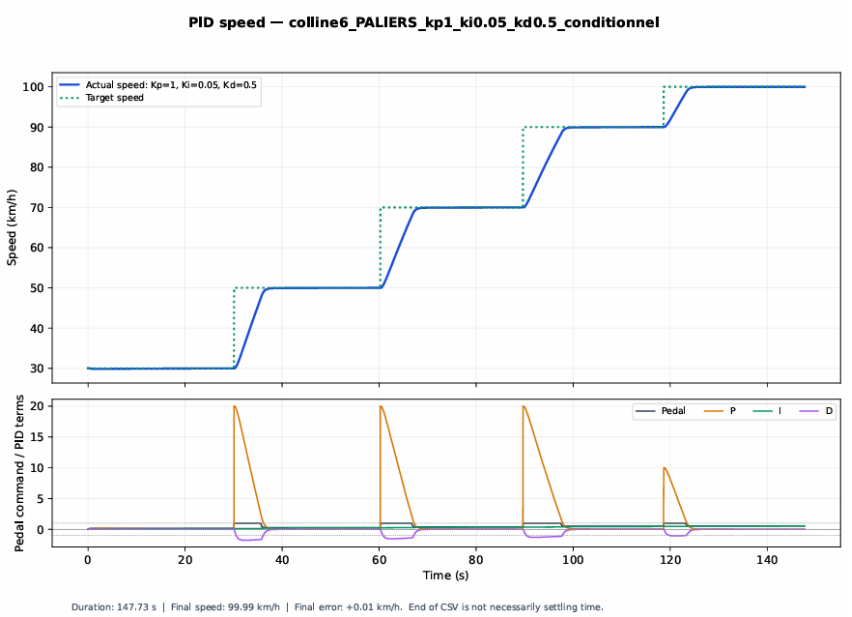
0.10 est essentiellement identique à 0.1063, donc il n’y a rien a observer.



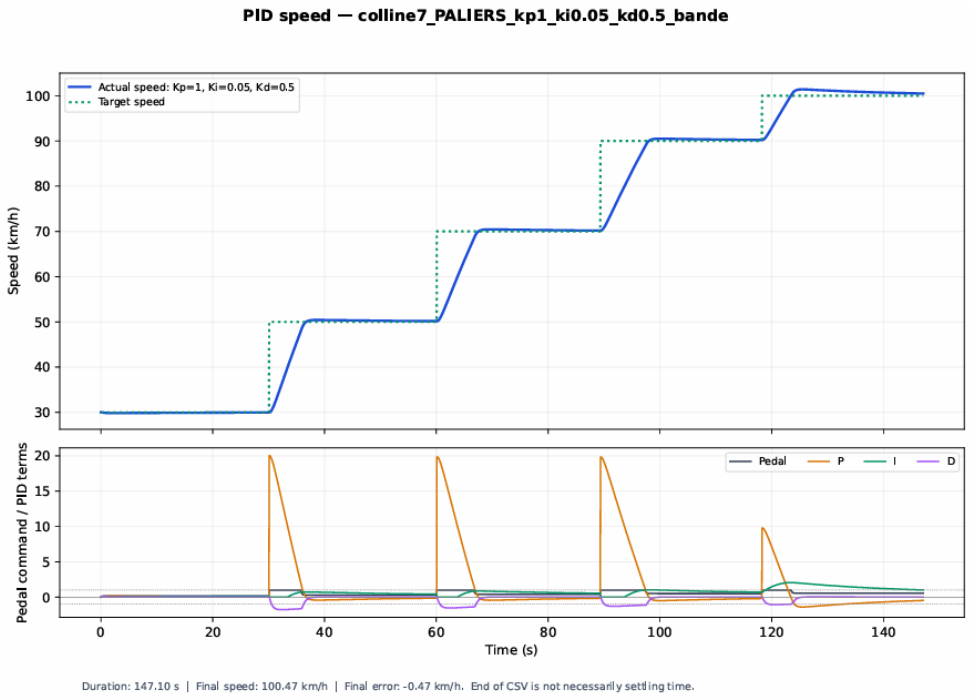
Lorsque la consigne de vitesse augmente, I remarque une erreur et voit qu'elle s'accumule tout au long de l'accélération de la voiture. Cette erreur accumulé cause un dépassement qui s'empire à chaque étape de vitesse car le I n'a pas le temps de corriger le gain énorme qu'il à accumulé lors de la montée.



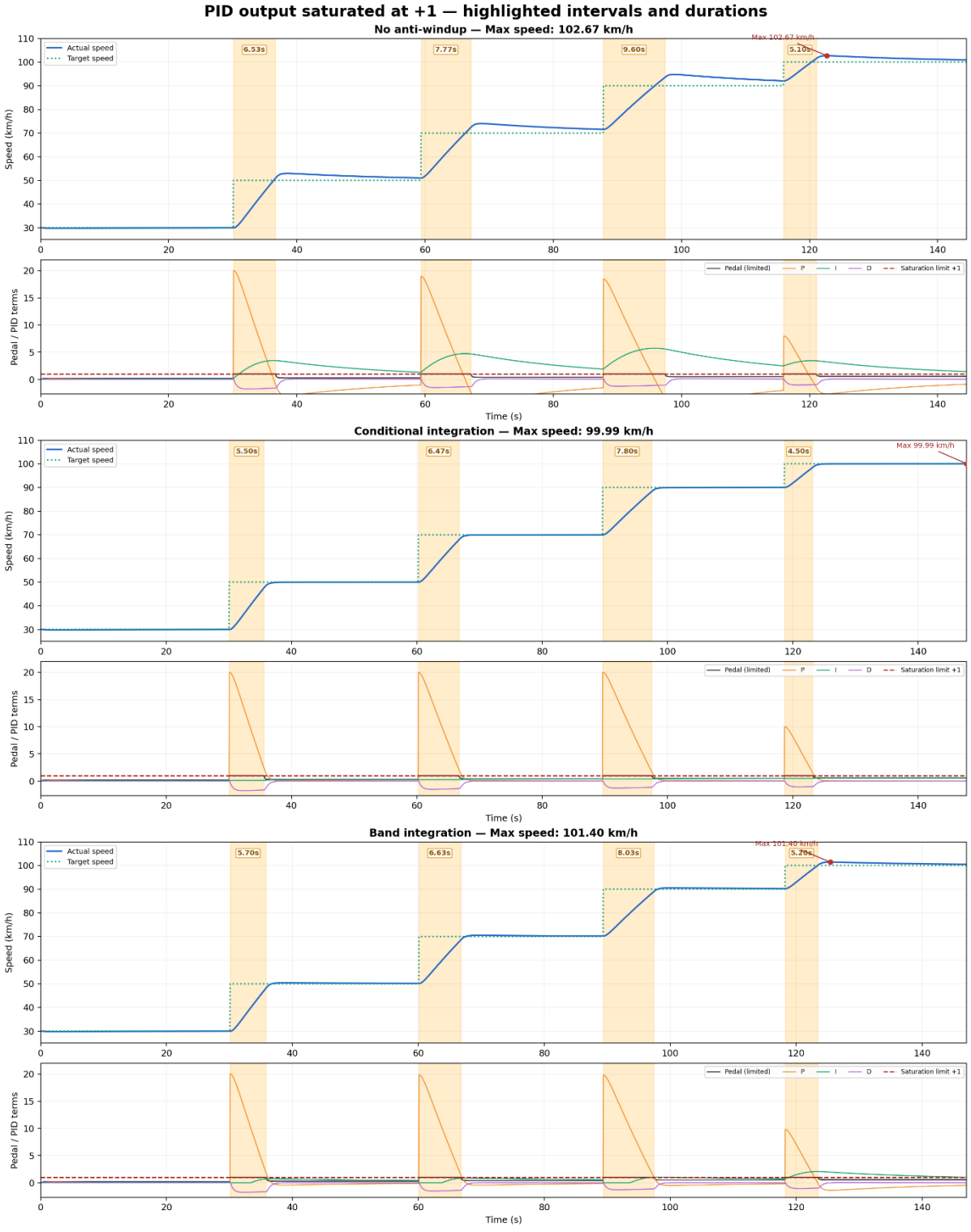
Ici, on arrête d'augmenter le gain en I si le gain PID dépasse +1 ou -1 afin que I n'accumule pas des erreurs qui ne servent à rien. En limitant l'intégrale, son gain n'augmente pas en continue durant la montée de vitesse et donc ne crée pas de dépassement une fois arrivé à la consigne.



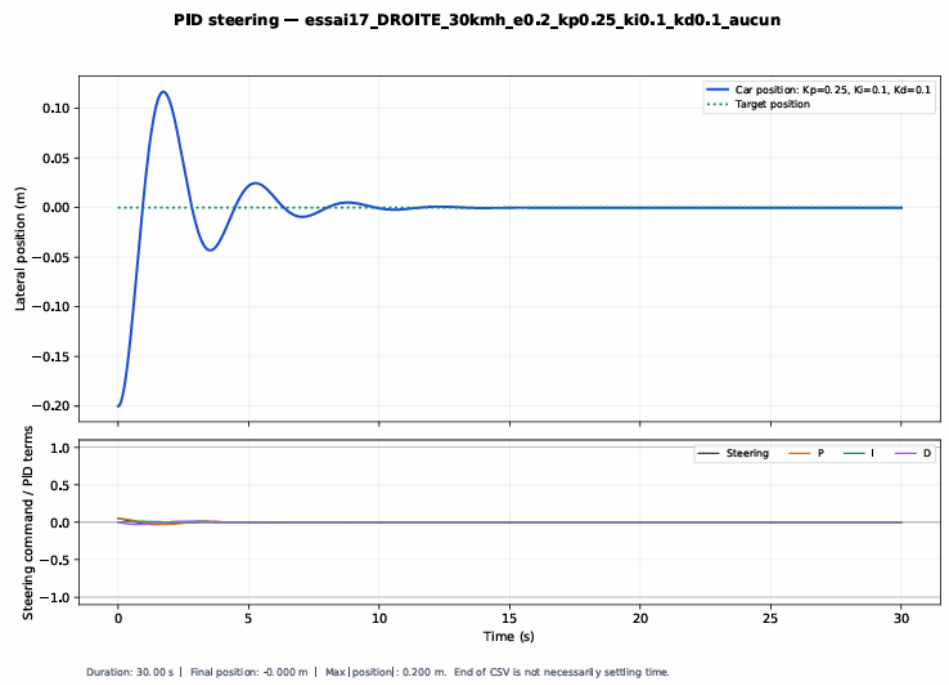
Dans le cas de la bande, c’est similaire, mais au lieu de limiter le I quand PID est saturé, on remettra I à 0 lorsque l’erreur est trop grande (pas dans la bande prédéfinie). Essentiellement, on sait qu’une erreur trop grande causera I à trop compenser et créer des dépassements donc lorsqu’une grande erreur a lieu, on enlève I et on laisse P et D régler la situation jusqu’à ce que l’erreur soit assez petite pour que l’intégrale participe sans créer des énormes gains inutiles.
On remarque ici qu’au dernier palier, le I continue quand même à influencer et donc crée un petit dépassement. C’est surement parce que le dernier palier est moins grand que les autres donc l’erreur qu’il crée n’est pas suffisante pour remettre le I à 0.



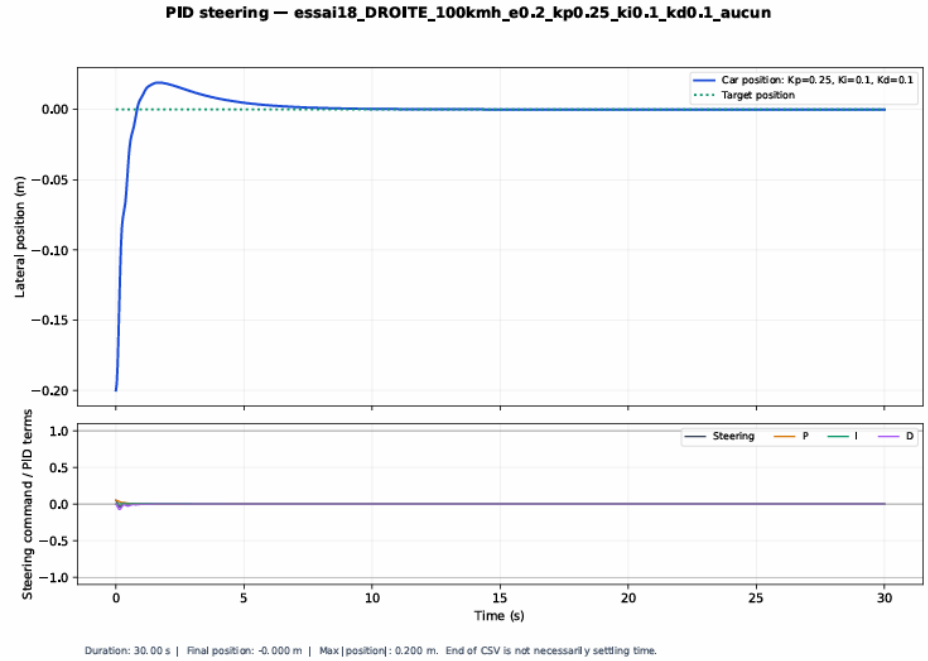
Ce graphique compare les 3 méthodes. Les zones en jaunes sont des moments où le PID = 1



Lorsque la voiture bouge lentement, on remarque des dépassements et l'oscillation. On peut raisonner que c'est en raison du gain intégrale qui accumule plus d'erreurs puisque la voiture prend plus longtemps à se corriger. Le I compense donc trop le gain puisqu'il croit que la voiture en a besoin.

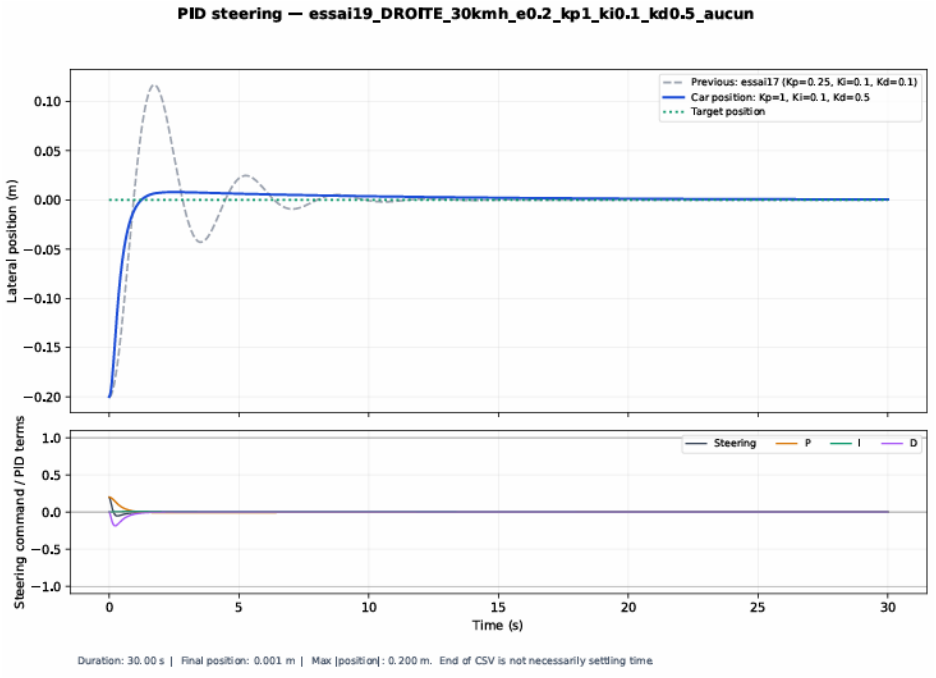


Lorsque la voiture bouge plus rapidement, l'oscillation disparait et on remarque un moins grand dépassement initialement. La voiture se corrige plus rapidement et donc le gain en I à moins le temps d'accumuler les erreurs et donc ne génère pas des dépassements.

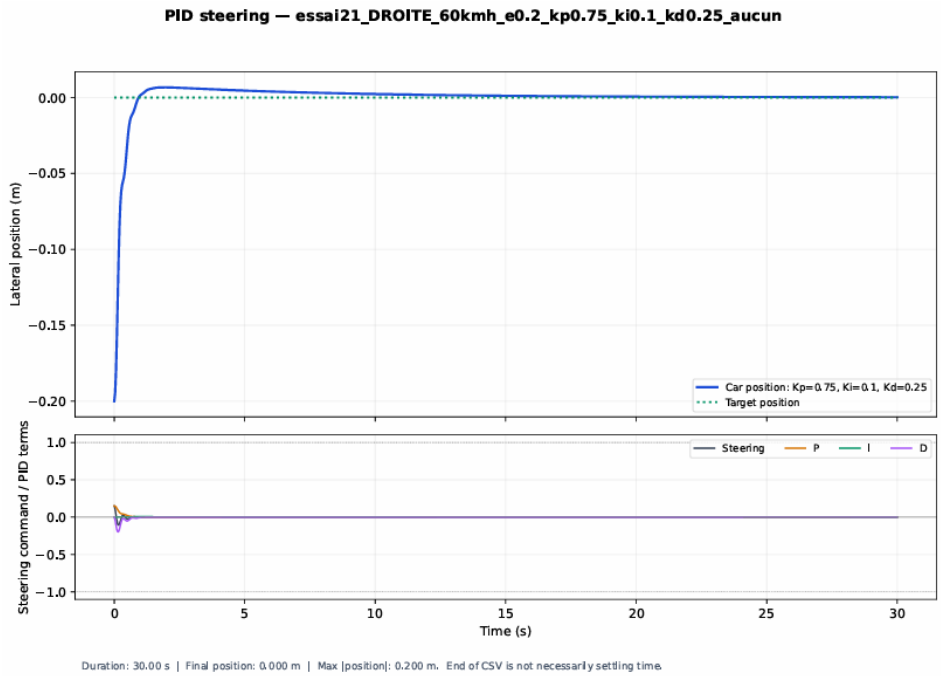


Voici les graphiques pour 30km, 60km et 90km ajustés :

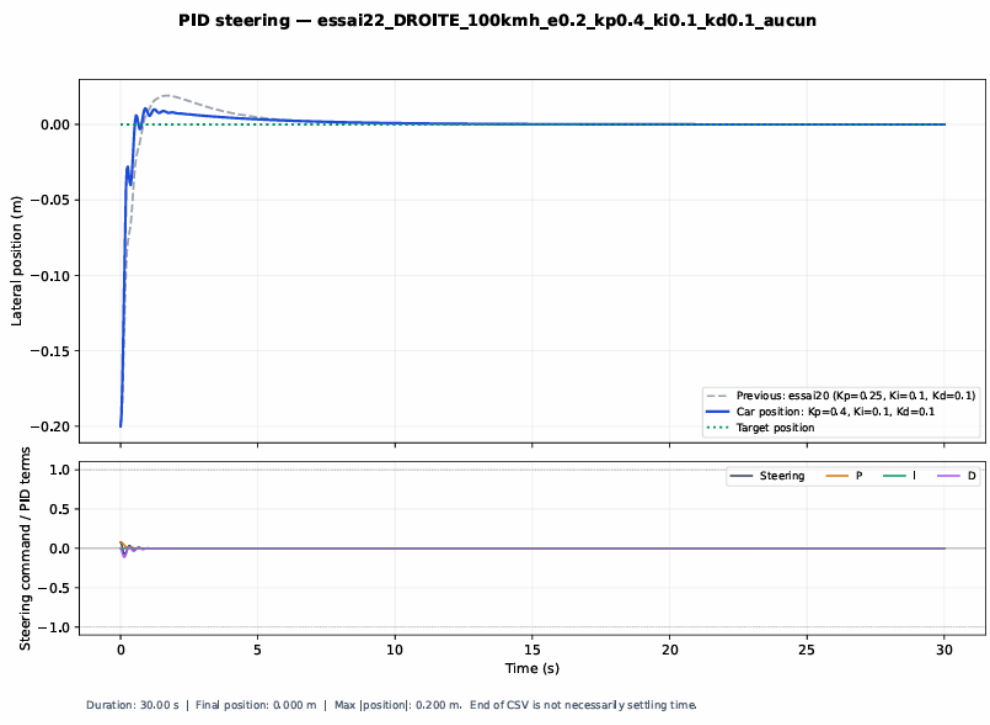
On a augmenté le gain en P pour réduire le temps que la voiture prend pour atteindre la consigne. On a également augmenté D pour atténuer les dépassements et par proximité, les oscillations.



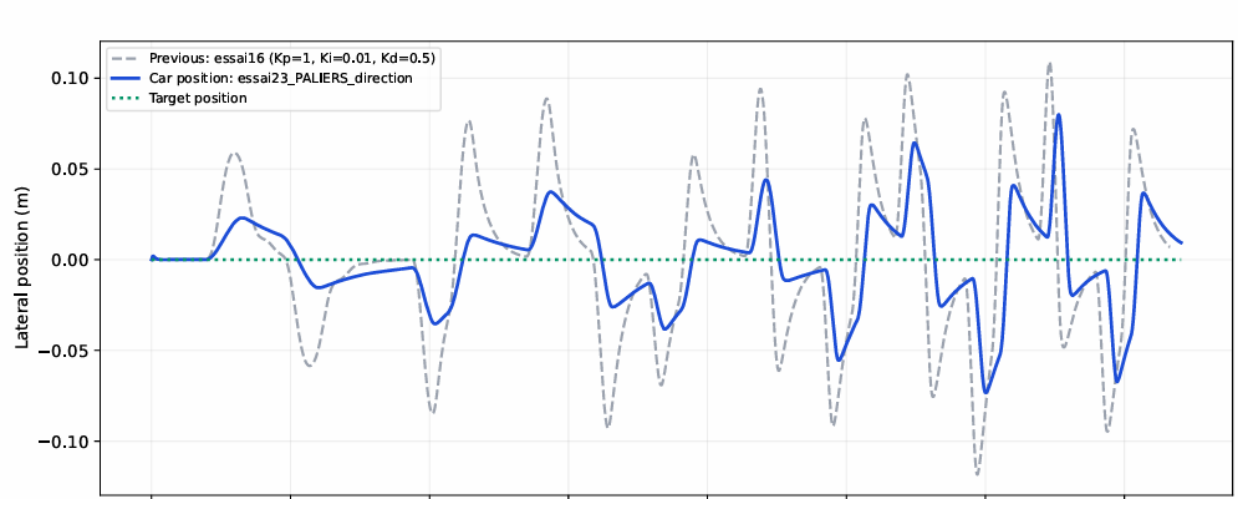
On augmente le P un peu pour accélérer les virements. Le reste peut rester identique.



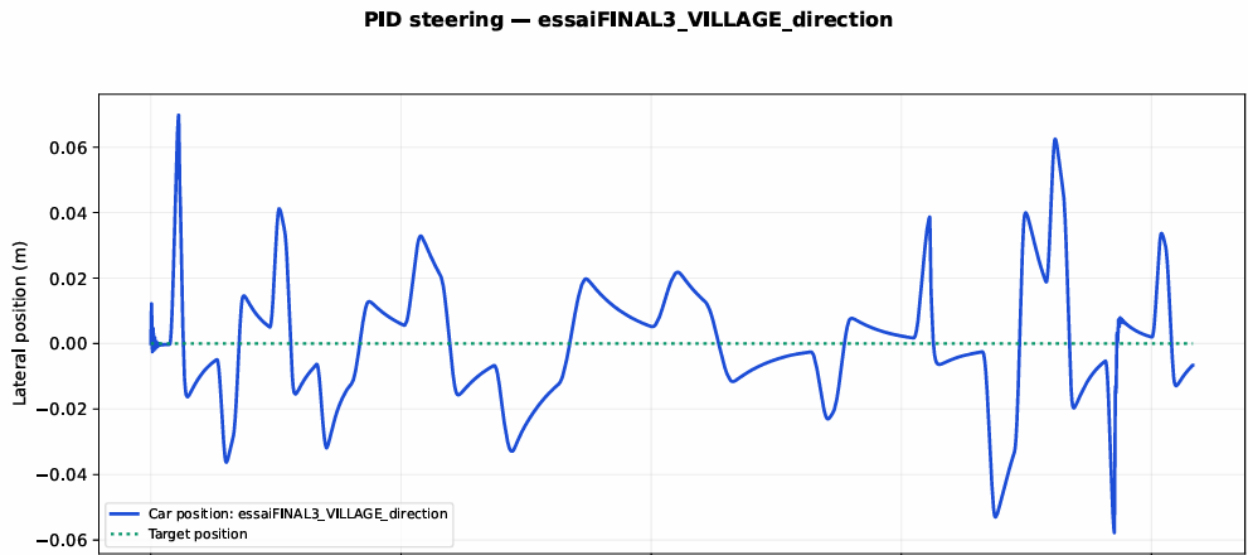
On diminue P pour réduire le dépassement. Un D plus élevé semble créer des oscillations et un D plus bas crée des dépassements. Malgré les petites oscillations au début, le système se stabilise très vite.

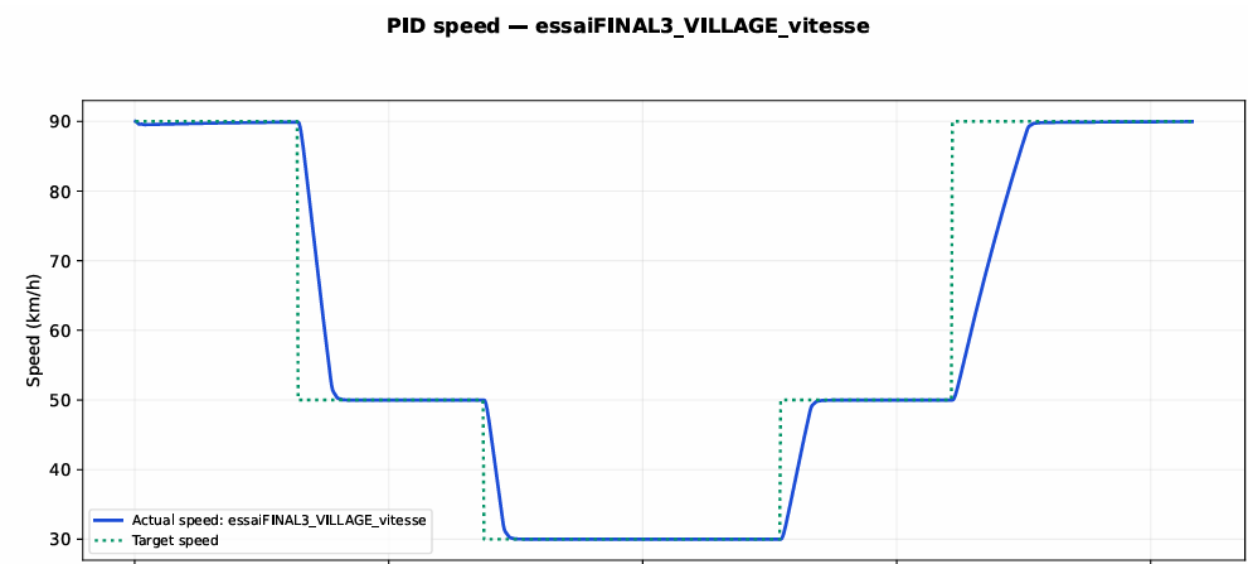


Voici l'essai avec les nouveaux paramètres dynamiques comparé avec l'essai de l'étape 4. On remarque une belle amélioration au niveau des piques. Les erreurs tout au long da la course sont moins grandes et la voiture se stabilise beaucoup plus vite que dans l'essai précédente. La basse vitesse cause les gains à être moins efficaces et en particulier, le gain intégral accumule plus d'erreurs et causes des oscillements. Au contraire, la vitesse très élevée accentue les corrections et causes des énormes oscillations lorsque le gain est trop élevé.



Course Finale : VILLAGE





COURSE FINALE : AUTOROUTE

